POLITECNICO DI MILANO

Software Engineering for Geoinformatics

Design Document

MeteoCity

City-Level Heat Monitoring using Meteostat

Academic Year 2025/26

Group 9

Authors:
Ayman Abdelgadir
Mobina Faraji
Hangyun Wang

June 2026

Table of Contents

List of Tables & List of Figures I

1. **Introduction** 1

1.1 Context and Motivations ……...1

1.2 Purpose ……..……...1

1.3 Scope & Limitations ……..2

1.4 Definitions and Acronyms …….……....3

1.5 Document Structure ……..3

2. Architecture design ……...…4

2.1 Overview ……...……...4

2.2 Component Diagram ..……...5

2.3 Technology Stack ……...……..7

3. Software Design ……..8

3.1 Overview …….…….…… 8

3.2 Database Design …….....…….…… 8

3.3 Backend Design ……...…….....9

3.3.1 API Endpoints ……..9

List of Tables

Table 1: Definitions and Acronyms …….3

Table 2: Database Tables Description ……....9

Table 3: REST API Endpoints ……10

List of Figures

Figure 1 : Three-tier Architecture ……..……...…….5

Figure 2 : Component Diagram ……...….6

Figure 3 : Database Schema Diagram ……...……...…8

1. Introduction

1. Context and Motivations

Public authorities and environmental agencies increasingly need reliable and accessible meteorological tools to understand and respond to the effects of rising temperatures in cities. Climate change is contributing to more frequent heat waves, warmer nights, and stronger heat-related stress, making urban weather monitoring critical for planners, local decision-makers, and researchers alike.

Weather-station data are particularly valuable in this context because they provide geographically explicit measurements that can be linked to real station locations and visualized on maps. Despite the availability of meteorological data sources such as Meteostat, accessible and role-specific platforms for retrieving, analyzing, and visualizing such data remain limited.

The MeteoCity system addresses this gap by providing:

- Retrieval and storage of historical hourly meteorological observations from Meteostat for selected cities.
- A structured relational database for persisting station metadata and weather observations.
- A REST API backend for querying and exposing processed meteorological data in JSON format.
- An interactive Jupyter Notebook dashboard for map-based and chart-based visualization and analysis.

1.2 Purpose

This Design Document (DD) translates the requirements defined in the Requirement Analysis and Specification Document (RASD) into a concrete technical design. It specifies:

- The three-tier architecture (data, application, presentation) and the interactions between components.
- The database schema optimized for structured meteorological storage.
- The REST API endpoints for data retrieval and processing.
- The integration strategy with the Meteostat API via RapidAPI.
- The main dashboard elements and user interface design.
- The implementation and testing plan.

1.3. Scope and Limitations

In Scope:

- Historical hourly meteorological observations for 2 selected cities from Meteostat.
- At least 2 weather data types (e.g., temperature and humidity).
- Role-based access for Environmental Officials, NGO Representatives, and Students.
- Interactive map-based and chart-based data visualizations.
- CSV data export functionality.

Out of Scope:

- Real-time weather forecasting or predictive modelling.
- Direct communication with physical meteorological stations.
- Alert or automated notification systems based on weather thresholds.
- Weather variables beyond those available through the Meteostat API.
- Mobile-specific interface design.

Limitations:

- The free Meteostat plan via RapidAPI is limited to 500 requests per month per user; requests must be managed carefully.
- Dependent on Meteostat API availability and data completeness.
- Geospatial queries may experience latency with large observation datasets.

1.4. Definitions and Acronyms

Table 1: Definitions and Acronyms

Term	Definition
API	Application Programming Interface
REST API	Representational State Transfer Application Programming Interface
GUI	Graphical User Interface
RASD	Requirement Analysis and Specification Document
DD	Design Document
JWT	JSON Web Token, used for secure authentication
PostGIS	PostgreSQL extension for geospatial data storage and queries
JSON	JavaScript Object Notation, used as data exchange format
RapidAPI	API marketplace used to access Meteostat data
ORM	Object-Relational Mapper (e.g., SQLAlchemy)
FR	Functional Requirement
NFR	Non-Functional Requirement
TR	Technical Requirement

1.5. Document Structure

This DD is organized as follows:

- Section 2: Architecture Design — three-tier structure and component diagram.
- Section 3: Software Design — database schema, REST API, Meteostat integration, and dashboard.
- Section 4: Implementation and Test Plan — development phases and test cases.

2. Architecture Design

1. Overview

The MeteoCity application is built on a robust, scalable three-tier architecture that separates concerns between data management, business logic, and user presentation. This design enables modular development, independent testing, and clear interfaces between layers, fulfilling TR2 (REST API backend) and TR3 (Jupyter Notebook dashboard) from the RASD.

The three tiers are defined as follows:

Presentation Tier

The client-facing interface implemented as a Jupyter Notebook dashboard. It provides interactive visualizations including maps, time-series charts, and summary statistics. Users interact with this layer to submit queries, apply filters, and export data. This tier communicates exclusively with the Application Tier via HTTP requests to the REST API.

Application Tier

The backend server implemented using Flask. It contains all business logic, handles user authentication, processes incoming requests, communicates with the Meteostat external API to ingest data, and interacts with the database for storage and retrieval. It exposes a set of REST API endpoints returning JSON-formatted responses.

Data Tier

The persistence layer composed of a PostgreSQL database extended with PostGIS for geospatial support. It stores station metadata, hourly meteorological observations, and user account information. Data is ingested from Meteostat and stored in structured, normalized tables to support efficient querying and visualization.

Figure 1 below illustrates this three-tier architecture in the context of the MeteoCity application.

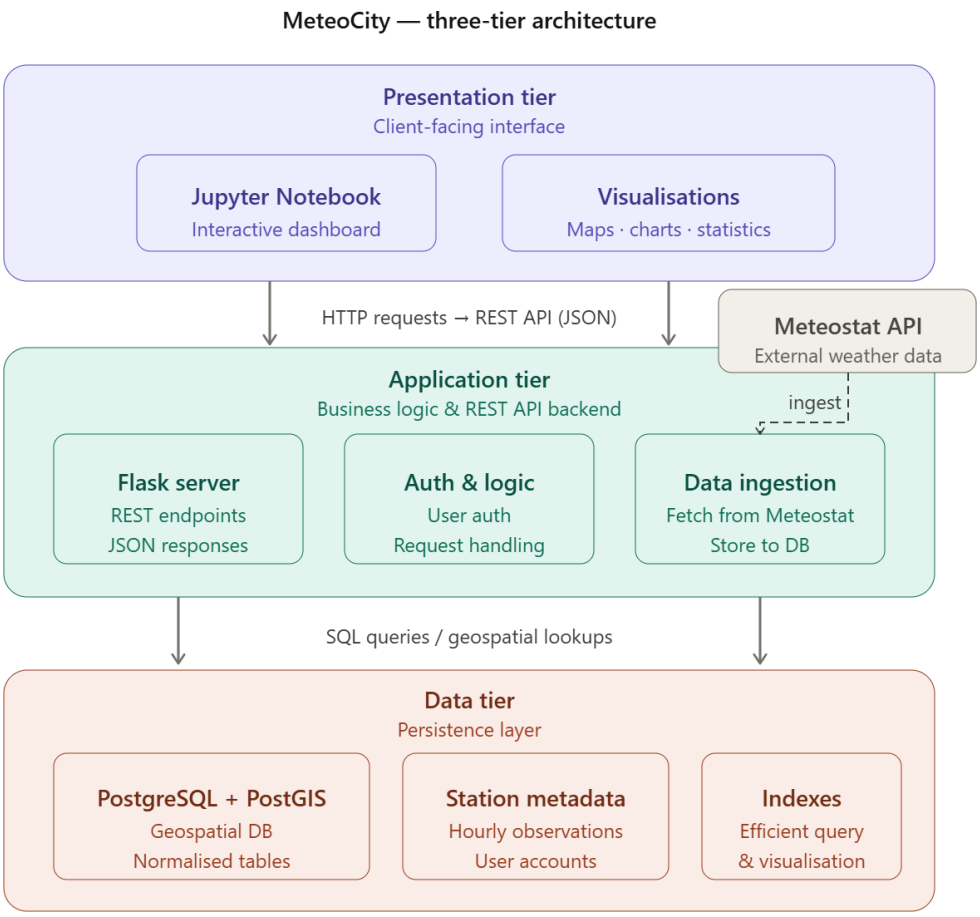


Figure 1: Three-tier architecture overview — MeteoCity application

2.2. Components Diagram

The system architecture is composed of several interconnected components working together to deliver the required functionality.

Figure 2 visualizes how the Jupyter Notebook dashboard (client) and the Flask application (server) are wholly separate, communicating only through the REST API. The database is populated via scheduled Python scripts that call the Meteostat RapidAPI.

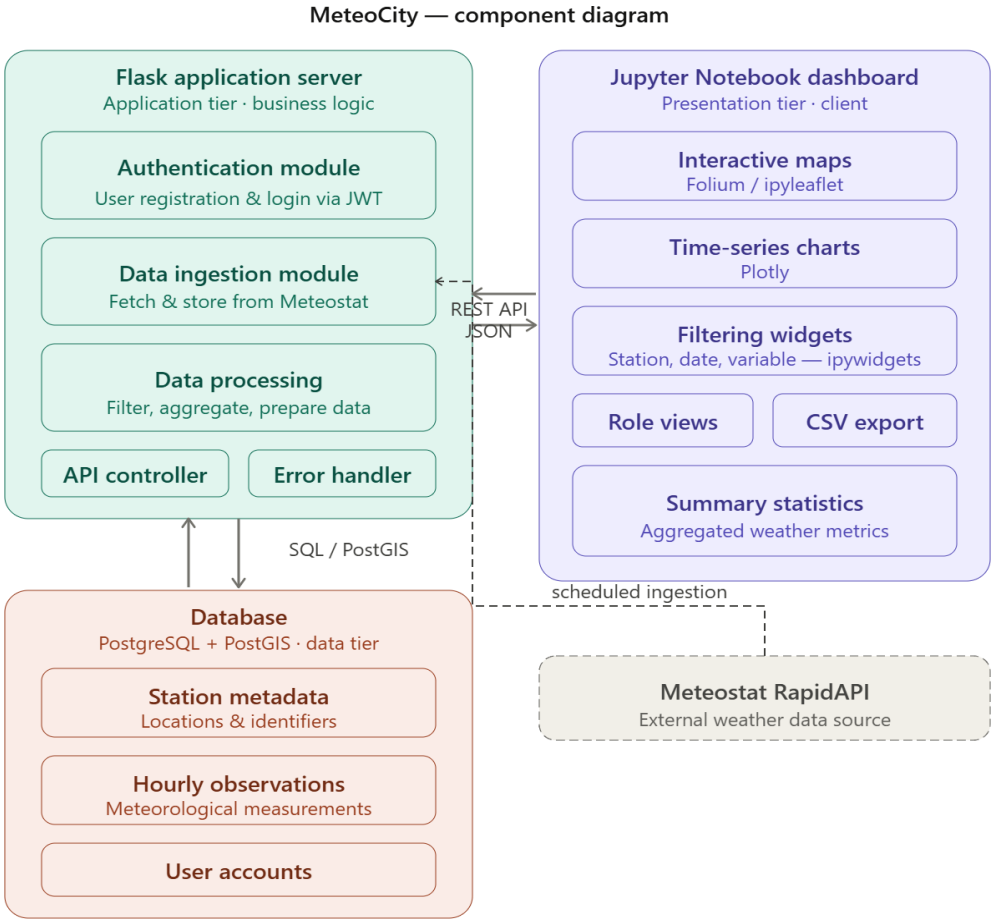


Figure 2: Component diagram of the MeteoCity application

The main components and their responsibilities are:

Flask Application (Server)

- Authentication Module: handles user registration and login via JWT tokens.
- Data Ingestion Module: interacts with the Meteostat RapidAPI to fetch and store station metadata and hourly observations.
- Data Processing Component: filters, aggregates, and prepares data for client consumption.
- API Controller: routes incoming HTTP requests to appropriate service handlers and returns JSON responses.
- Error Handler: logs errors and monitors API request success and failure rates.

Database (PostgreSQL + PostGIS)

- Stores user account data, weather station metadata, and hourly meteorological observations.
- Receives data from the Meteostat API via scheduled ingestion scripts.
- Supports geospatial queries via the PostGIS extension.

Jupyter Notebook Dashboard (Client)

- Presents interactive maps using Folium or IpyLeaflet.
- Renders time-series charts using Plotly.
- Provides filtering widgets (station, date range, weather variable) using ipywidgets.
- Supports role-specific views via Mercury.
- Allows CSV data export.

2.3. Technology Stack

The following technology choices directly support the functional and non-functional requirements defined in the RASD:

Database

- PostgreSQL with PostGIS extension for storing station metadata, geospatial coordinates, and time-series observations (supports TR5: structured meteorological storage).
- Scheduled Python scripts for data ingestion from Meteostat via RapidAPI (supports TR1: Meteostat API integration).

Backend

- Flask server exposing REST API endpoints in JSON format (supports TR2 and TR6).
- JWT authentication for secure, role-based access control (supports FR access levels).
- SQLAlchemy ORM for flexible and secure database query construction.
- GeoJSON responses for map-based data delivery.

Dashboard

- Jupyter Notebook as the core dashboard platform (supports TR3).
- Folium or IpyLeaflet for interactive geographic maps.
- Plotly for time-series and summary charts.
- ipywidgets for user-driven filtering (city, date range, variable).
- Mercury for role-based view rendering (Environmental Official, NGO, Student). And CSV export functionality

3. Software Design

3.1 Overview

The MeteoCity software is designed to support meteorological data retrieval, storage, and analysis using the Meteostat historical dataset. It is composed of three main software components: a PostgreSQL database, a Flask-based web server (backend), and a Jupyter Notebook dashboard (frontend). Each component has a well-defined role, and they interact through REST API calls and structured SQL queries to provide a seamless user experience.

3.2 Database Design

The database serves as the backbone of the system, responsible for persisting all station metadata and hourly meteorological observations retrieved from Meteostat. A well-structured schema is critical to support efficient querying, geospatial filtering, and reliable data delivery to the dashboard.

The schema is designed to be normalized, ensuring data integrity and avoiding redundancy. The PostGIS extension is used to store and query geographic coordinates associated with weather stations. Figure 3 provides the entity-relationship diagram.

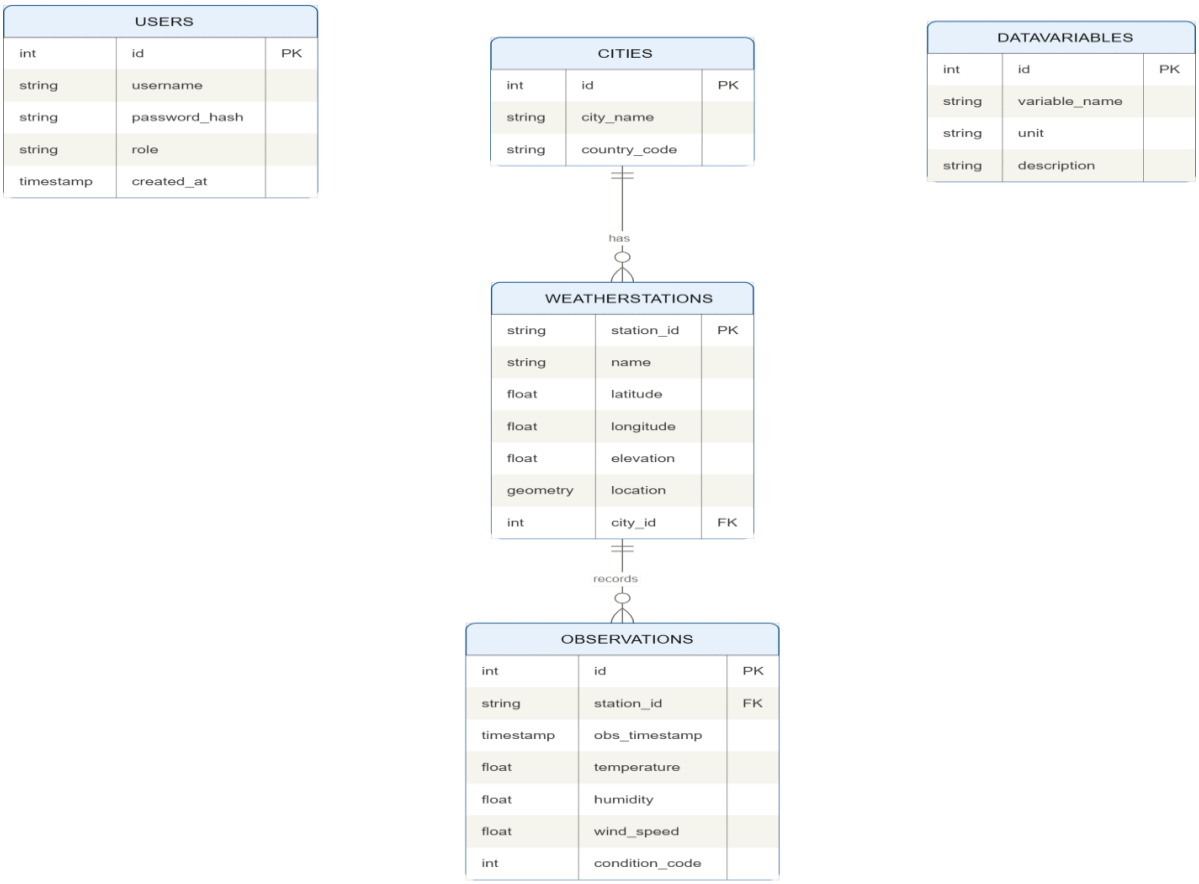


Figure 3: Database schema diagram — MeteoCity application

Table 2 describes each table in the database schema.

Table 2: Database Tables Description

Table Name	Description	RASD Requirements
Users	Stores user account data including username, hashed password, and role (student, ngo_company, government). Determines access permissions within the application.	FR (Access Control)
Cities	Stores the list of monitored cities including city name and country code. Serves as a reference for station-to-city association.	FR1, FR3
WeatherStations	Stores station metadata retrieved from Meteostat, including station ID, name, latitude, longitude, elevation, and city association. Uses PostGIS for geospatial storage.	FR2, TR1, TR5
Observations	Stores hourly meteorological observations for each station. Fields include station_id (FK), timestamp, temperature (°C), humidity (%), wind speed (km/h), and weather condition code. Linked to WeatherStations.	FR1, FR2, FR3, FR5
DataVariables	Defines the meteorological variables tracked by the system (e.g., temperature, humidity, wind speed). Includes variable name, unit, and description.	FR6, NFR2

The primary relationships are as follows:

- Each WeatherStation belongs to one City (many-to-one).
- Each Observation is linked to one WeatherStation via station_id (foreign key).
- Users are independent of the geospatial data model; their role determines API access.

3.3 Backend Design

The backend is a Flask application responsible for exposing the REST API, handling all data ingestion from Meteostat, processing user requests, and returning structured JSON responses. It implements JWT-based authentication for role-based access control.

3.3.1 API Endpoints

Table 3 lists all REST API endpoints exposed by the MeteoCity backend.

Table 3: REST API Endpoints

Method	URL	Description	Accepted Input	RASD Ref.
POST	/auth/register	Registers a new user account	username, password, role	FR (Users)
POST	/auth/login	Authenticates user and returns JWT token	username, password	FR (Users)
GET	/auth/logout	Logs out the current user	None	FR (Users)
GET	/auth/profile	Returns current user profile and role	JWT token	FR (Users)
GET	/api/cities	Returns the list of monitored cities	None	FR3
GET	/api/stations	Returns weather stations with metadata and geolocation	city_name (optional)	FR2, TR5
GET	/api/observations	Returns hourly meteorological observations with filters	station_id, start_date, end_date, variable	FR1, FR2, FR3
GET	/api/trend_data	Returns aggregated time-series data for trend analysis	station_id, start_date, end_date, variable	FR5, FR7
GET	/api/statistics	Returns statistical summary (mean, max, min, std) for a selected period	station_id, start_date, end_date, variable	FR5
GET	/api/map_data	Returns GeoJSON-formatted station data with observation summaries for map display	city_name, start_date, end_date	FR4
GET	/api/export_csv	Exports filtered observations as a downloadable CSV file	station_id, start_date, end_date, variable	TR7
GET	/api/variables	Returns the list of available meteorological variables	None	FR6